Workflow Engines

durable execution: code that survives crashes, waits and people

Watch the 4-min video

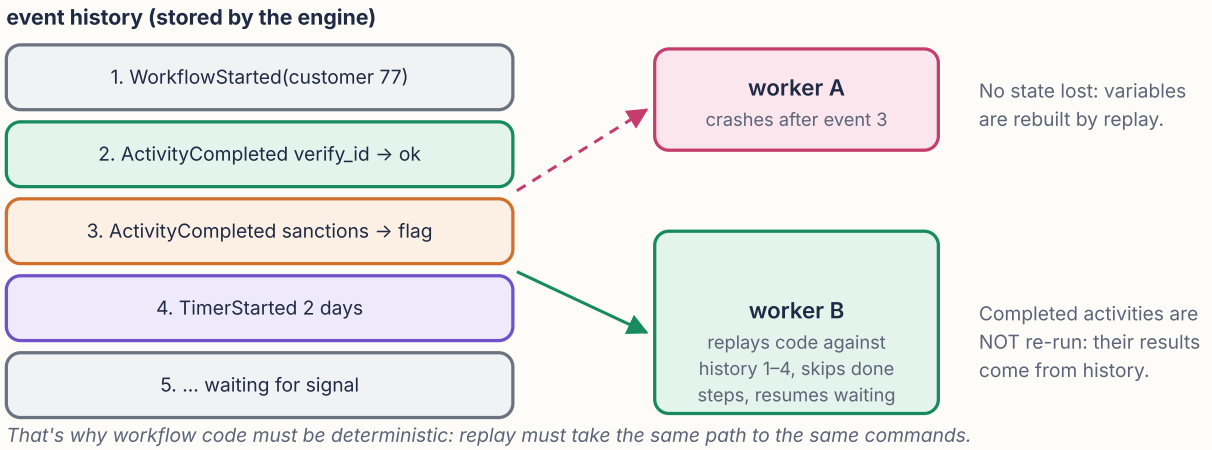
In one line: a workflow engine runs a multi-step business process as ordinary-looking code whose progress is recorded step by step, so it survives crashes, retries steps automatically, can wait for timers or human signals for days, and runs compensations; the price is that workflow code must be deterministic, with side effects in activities.

1 Where were we?

Opening a bank account: verify identity with a third-party service, run a sanctions check, wait for a compliance officer (which can take two days), open the account, send a welcome email. If the service restarts in the middle, where were we? Built from queues and cron jobs, this becomes a status table, a retry loop per step, a timer job that scans for stuck rows, and a dashboard nobody trusts. Workflow engines make the process itself durable.

2 Durable execution

You write the workflow as a function. The engine (Temporal, for example) records an **event history** for each run: started, each activity scheduled and completed with its result, timers, signals. If a worker crashes, another worker picks the run up and **replays** the function from the beginning against that history: completed steps return their recorded results instantly instead of running again, so the code reaches the exact point where it stopped, with all local variables rebuilt. A run can last seconds or months.



That's why workflow code must be deterministic: replay must take the same path to the same commands.

Figure 1. Replay: the history is the source of truth; the code is re-executed to rebuild state, and only new steps actually run.

3 Workflows and activities

Activities are the steps that touch the outside world: call the identity provider, write to a database, send an email. The engine gives each one **timeouts** (for example start-to-close, schedule-to-close, and heartbeats for long tasks) and a **retry policy** with exponential backoff (Temporal's default starts at 1 s, doubles each time, and retries until a timeout or a non-retryable error). Because an activity can run more

than once (a worker can finish the call and crash before reporting it), activities should be **idempotent**, for example by passing an idempotency key to the provider.

Workflow code orchestrates and must be **deterministic**: given the same history it must issue the same commands in the same order. So no direct network or database calls, no system clock (use the engine's `workflow.now()`), no random numbers or UUIDs except through engine APIs, no iterating over unordered maps where order matters. Changing workflow code while runs are in flight needs **versioning** (patch markers), or replay of old runs will diverge.

4 Long waits are first-class

A workflow can `sleep(30 days)`, set a timer, or wait for a **signal** (an external message such as "officer approved") without holding a thread, a process or a connection: the engine stores a timer or waits for an event, and the run is simply resumed when it fires. That makes human-in-the-loop steps, reminders, escalations and deadlines a few lines of code. Runs can also be **queried** for their current state, which is what support staff see in the UI.

```
@workflow.defn
class Onboarding:
    def __init__(self): self.decision = None

    @workflow.signal
    def review(self, decision: str): self.decision = decision

    @workflow.run
    async def run(self, c: Customer):
        await workflow.execute_activity(verify_id, c, start_to_close_timeout=MIN5,
                                        retry_policy=RetryPolicy(maximum_attempts=10))
        risk = await workflow.execute_activity(sanctions, c, start_to_close_timeout=MIN5)
        if risk.flagged:
            await workflow.execute_activity(create_review_task, c, ...)
            try:   # durable wait: no thread is held for these 2 days
                await workflow.wait_condition(lambda: self.decision, timeout=DAYS2)
            except asyncio.TimeoutError:
                await workflow.execute_activity(escalate, c, ...)
                await workflow.wait_condition(lambda: self.decision)
            if self.decision != 'approve': return await reject(c)
```

Figure 2. Temporal-style Python (simplified). Activities do the I/O with timeouts and retries; the workflow just waits for a signal with a 2-day deadline.

5 Engines and when to use one

Engine	Model	Good for
Temporal (and Cadence, its Uber predecessor)	workflows as code in Go, Java, Python, TypeScript, .NET; event history + replay	long-running business processes, sagas, high volume
AWS Step Functions	JSON state machine (Amazon States Language); Standard (up to a year) and Express (up to 5 minutes)	orchestrating AWS services; callbacks with task tokens
Azure Durable Functions	orchestrator functions with replay, durable timers, external events	Azure serverless processes
Apache Airflow (also Dagster, Prefect)	scheduled DAGs of tasks	batch data pipelines, not per-customer request flows
Camunda, Conductor	BPMN diagrams / JSON workflow definitions	processes designed with business analysts; microservice orchestration

Use one when a process has many steps, must survive failures and restarts, includes long waits or people, needs **compensation** when a later step fails (a saga), or must be visible and auditable step by step: support can see exactly which step customer 77 is stuck on, and why.

6 When a queue is enough

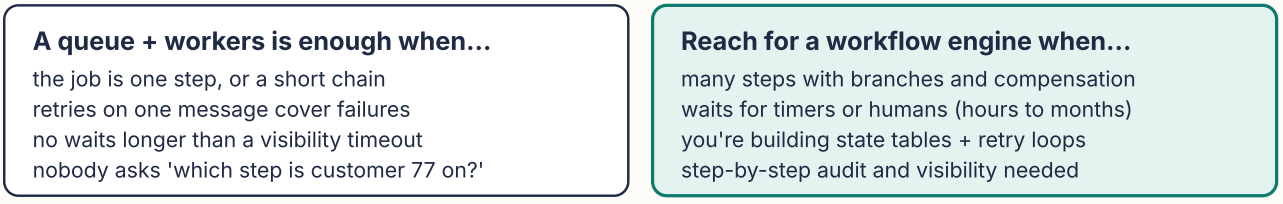


Figure 3. The tell-tale sign: you're writing your own state tables, retry loops and timers around a queue.

Engines add infrastructure to operate (or a managed service to pay for), a programming model to learn (determinism, versioning), and some latency per step (each step is a durable write). For a single background job such as resizing an image or sending one email, a queue with retries and a dead-letter queue is simpler and cheaper.

7 The KYC onboarding workflow

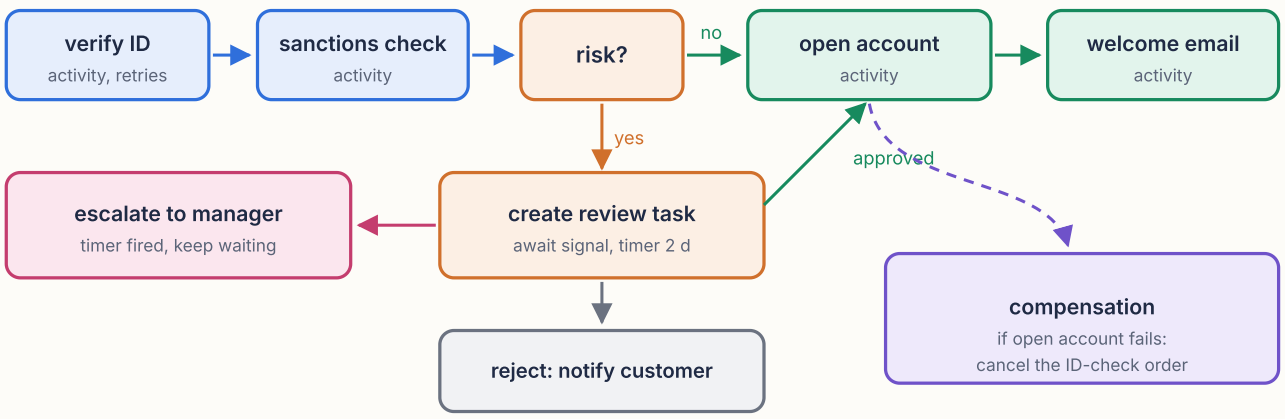


Figure 4. The worked example's flow. Every arrow is a durable step; a crash anywhere resumes at the same point.

WORKED EXAMPLE: KYC ONBOARDING WITH MANUAL REVIEW

The workflow calls the **verify-identity** activity with retries and an idempotency key (so a retried call doesn't create a second paid check), then the **sanctions** activity. If either flags a risk, it creates a review task in the compliance tool and **waits for a signal**, with a **2-day timer**: if the timer fires first, it escalates to a manager and keeps waiting. When the officer clicks approve, the tool sends the signal; the workflow wakes, **opens the account** and sends the welcome email. If opening the account fails permanently, a **compensation** step cancels the identity verification order (and anything else already done, in reverse order). If the worker host is replaced on day one of the wait, nothing happens at all: the run is just a history and a timer until the signal arrives.

COMMON MISTAKES

- Calling APIs or reading the clock directly in workflow code, breaking replay.
- Non-idempotent activities that double-charge or double-send on retry.
- Deploying changed workflow code without versioning while old runs are in flight.
- Using Airflow for per-request business processes, or Temporal for one-step jobs.

8 Key terms

Durable execution: progress persisted so code resumes after any crash

Event history: recorded log of a run's steps and results

Activity: a step with side effects, retried with timeouts

Determinism: same history → same commands on replay

Signal: external message delivered to a running workflow

Compensation: undo action for an earlier step (saga)

9 Interview questions

Q1. When would you use a workflow engine instead of queues? BASIC

When a business process has many steps, must survive crashes and restarts, includes long waits, timers or human approvals, needs compensation, or must be auditable step by step; durable execution handles state, retries and timers for you.

Q2. What's the key rule for workflow code? BASIC

It must be deterministic, with all side effects (I/O, time, randomness) in activities or engine APIs, because the engine replays it from history.

Q3. What is a signal? BASIC

A message sent to a running workflow from outside, such as a human approval, that it can wait on durably.

Q4. A worker crashes after the identity check succeeded. What happens? SCENARIO

Another worker replays the workflow; the identity check's result comes from history, so it isn't called again, and execution continues with the sanctions check.

Q5. How do you implement 'remind the user after 3 days, cancel after 7'? SCENARIO

A workflow with durable timers: sleep 3 days, send the reminder activity unless a completion signal arrived, then wait up to 4 more days before running the cancel activity.

Q6. Can an activity run twice? What do you do about it? TRICKY

Yes: a worker can complete it and crash before the result is recorded, so it's retried; make activities idempotent with idempotency keys or upserts.

Q7. Why can't you just add a new step in the middle of the workflow code and deploy? TRICKY

In-flight runs replay against histories that don't contain that step, so replay diverges; use versioning (patch markers) so old runs follow the old path.

Next: SD-V-022 · Distributed Job Scheduling backendroadmap.com